

M1 ILSEN • 2026-2027

Techniques de tests

CM 2 • De la théorie à la pratique

QUESTION DIRECTRICE

Comment vérifier qu'un logiciel se comporte comme prévu ?



Chaque test vérifie une attente précise dans un contexte donné.

À LA FIN DE CETTE SÉANCE

Vous devriez pouvoir...

1

Écrire

Des tests unitaires lisibles avec **pytest**.

2

Explorer

Formuler une propriété et générer automatiquement des cas de test.

3

Isoler

Remplacer certaines dépendances sans tout simuler.

4

Concevoir

Rendre les dépendances contrôlables et les effets observables.

5

Mesurer

Interpréter performance et couverture avec recul.

6

Automatiser

Organiser une vérification locale reproductible.

RAPPEL

TDD : définition et apports

Un rappel du principe, de la boucle et de ses limites.

TDD en une idée

**Écrire l'exemple qui manque
avant le code qui le satisfait.**

Ce que cela apporte

- un objectif observable
- un feedback immédiat
- un comportement traité à la fois

Ce que cela n'apporte pas seul

- le bon besoin
- une bonne architecture
- une garantie d'exhaustivité

Red · Green · Refactor



Red

Écrire un test qui échoue pour la bonne raison.



Green

Produire le code minimal qui satisfait l'attendu.



Refactor

Améliorer la structure sans changer le comportement.

Si le test ne passe jamais par le rouge, sait-on vraiment ce qu'il détecte ?

RED

Commencer par un comportement

```
def test_rejects_an_empty_username():  
    assert is_valid_username("") is False
```

Le test échoue parce que `is_valid_username` n'existe pas encore.

Le nom du test décrit une règle métier, pas une méthode interne.

GREEN

Faire passer le test, simplement

```
def is_valid_username(username: str) -> bool:  
    return username != ""
```

Maintenant

Le comportement demandé est présent.

Pas encore

Les espaces seuls ne sont pas traités.

REFACTOR

Généraliser juste assez

Nouveau cas

```
def test_rejects_only_spaces():  
    assert is_valid_username(" ") is False
```

Implémentation

```
def is_valid_username(username: str) -> bool:  
    return bool(username.strip())
```

Le cycle recommence dès qu'un nouveau comportement utile est identifié.

TDD : le rappel essentiel

À conserver

- des itérations courtes
- un test observé en échec
- une implémentation minimale
- du refactoring régulier

À éviter

- écrire tous les tests mécaniquement
- coupler les tests aux détails internes
- confondre tests passants et bon design
- traiter TDD comme une religion

TDD organise une boucle de feedback. Il ne remplace pas le jugement.

TESTS UNITAIRES

Tests unitaires avec pytest

Tester rapidement un comportement isolé.

Une « unité » est une frontière utile

Fonction

Une transformation pure et ses cas limites.

Objet

Un comportement public et son état observable.

Petit composant

Une responsabilité avec ses dépendances contrôlées.

L'unité n'est pas nécessairement une ligne, une méthode ou une classe.

Un bon test unitaire est...

1

Lisible

Son intention est immédiate.

2

Déterministe

Même contexte, même résultat.

3

Rapide

Il peut être exécuté souvent.

4

Isolé

Il localise clairement l'échec.

Un bon test vérifie un comportement précis et facilite le diagnostic en cas d'échec.

Arrange · Act · Assert

Arrange

Préparer le contexte



Act

Exécuter le comportement



Assert

Comparer à l'attendu

```
def test_overlapping_periods_conflict():  
    existing = Period(start=9, end=11)    # Arrange  
    requested = Period(start=10, end=12)  
    conflict = overlaps(existing, requested) # Act  
    assert conflict is True                # Assert
```

Un premier test avec pytest

Code testé

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Period:
    start: int
    end: int

    def __post_init__(self):
        if self.end <= self.start:
            raise ValueError("invalid period")

def overlaps(first: Period, second: Period) -> bool:
    return first.start < second.end and second.start < first.end
```

Test

```
def test_overlapping_periods_conflict():
    existing = Period(9, 11)
    requested = Period(10, 12)

    assert overlaps(existing, requested)
```

```
$ pytest -q
1 passed in 0.02s
```

`-q` signifie « quiet » : pytest affiche un résultat plus concis.

Tester le comportement public

Comportement observable

```
calendar.add(existing)  
assert calendar.is_available(requested) is False
```

Le test utilise l'API publique.

Détail interne

```
calendar.add(existing)  
assert calendar._reservations == [existing]
```

Le test dépend du stockage choisi.

Un refactoring peut changer le stockage interne sans changer la disponibilité annoncée.

Paramétrer les variations

```
import pytest

@pytest.mark.parametrize(
    ("requested", "expected"),
    [
        (Period(8, 9), False),
        (Period(8, 10), True),
        (Period(10, 12), True),
        (Period(11, 12), False),
    ],
)
def test_overlap_with_period_from_9_to_11(requested, expected):
    assert overlaps(Period(9, 11), requested) is expected
```

Les cas avant, après et exactement sur les frontières restent visibles.

Les erreurs font partie du contrat

```
import pytest

def test_rejects_a_period_ending_before_it_starts():
    with pytest.raises(ValueError):
        Period(start=11, end=9)
```

Les entrées invalides et les erreurs attendues font partie du comportement à définir.

Une fixture prépare un contexte réutilisable

Préparation

```
import pytest

@pytest.fixture
def empty_cart():
    return Cart(currency="EUR")
```

Utilisation

```
def test_new_cart_has_no_items(empty_cart):
    assert empty_cart.items == []
    assert empty_cart.total == 0
```

pytest injecte la fixture en faisant correspondre son nom au paramètre du test.

La portée d'une fixture est un choix d'isolation

Portée	Création	Usage
fonction	pour chaque test	état mutable, choix par défaut
module	pour un fichier	ressource coûteuse et maîtrisée
session	pour toute l'exécution	infrastructure stable

Partager une ressource peut accélérer les tests. Partager son état peut les rendre interdépendants.

Tester une API sans lancer un vrai serveur

Application Flask

```
@app.get("/status")
def status():
    return {"status": "ready"}, 200
```

Client de test

```
def test_status_endpoint(client):
    response = client.get("/status")

    assert response.status_code == 200
    assert response.json == {"status": "ready"}
```

Le test traverse le routage et la sérialisation, sans réseau réel.

Choisir le niveau qui répond à la question

Question	Niveau plausible
Le calcul respecte-t-il cette règle ?	fonction ou objet
Deux périodes qui se touchent sont-elles compatibles ?	fonction de comparaison
L'API renvoie-t-elle le bon statut ?	client Flask
Deux services communiquent-ils réellement ?	intégration

Commencer par le test le plus petit qui répond à la question. Ajouter un test plus large lorsqu'une interaction doit réellement être vérifiée.

Contrôler les sources d'instabilité

Temps

Éviter l'heure réelle quand elle influence le résultat.

Hasard

Fixer une graine ou conserver l'exemple qui échoue.

Réseau

Réserver les appels réels aux tests qui en ont besoin.

Fichiers

Utiliser un répertoire temporaire propre.

Configuration

Contrôler les variables et options de l'environnement.

État partagé et ordre

Chaque test doit pouvoir être exécuté seul.

OUVERTURE, HORS PÉRIMÈTRE DU TP

Tester des propriétés, pas seulement des exemples

Property-based testing avec Hypothesis.

Un exemple vérifie un cas précis

```
def test_reverse_three_values():  
    assert reverse([1, 2, 3]) == [3, 2, 1]
```

Entrée choisie → Exécution → Résultat attendu

Le test communique clairement un cas, mais ne dit rien sur toutes les autres listes.

Une propriété décrit une relation générale

**Inverser deux fois
redonne la valeur initiale.**

```
reverse(reverse(values)) == values
```

Une propriété décrit une relation générale, sans calculer séparément le résultat de chaque exemple.

Hypothesis génère les exemples

```
from hypothesis import given
from hypothesis import strategies as st

@given(st.lists(st.integers()))
def test_reversing_twice_returns_the_original(values):
    assert reverse(reverse(values)) == values
```

Générer



Exécuter



**Trouver un contre-
exemple**



Le simplifier

Une fonction peut avoir plusieurs propriétés

Ordre

Chaque valeur est inférieure ou égale à la suivante.

Conservation

Les valeurs et leurs occurrences sont préservées.

Idempotence

Trier un résultat trié ne le change pas.

```
@given(st.lists(st.integers()))
def test_sort_is_idempotent(values):
    assert sort(sort(values)) == sort(values)
```

Une fonction qui renvoie toujours `[]` est idempotente. Une seule propriété ne suffit donc pas à caractériser un tri.

Shrinking : chercher le petit contre-exemple

Propriété

```
@given(st.lists(st.text()))
def test_round_trip(parts):
    assert decode(encode(parts)) == parts
```

Échec réduit

```
Falsifying example:
parts=[]

encode([]) == ''
decode('') == []
```

Le meilleur contre-exemple n'est pas le plus spectaculaire. C'est celui que l'on comprend vite.

Construire ou filtrer le domaine

Construire les valeurs pertinentes

```
identifiers = st.text(  
    alphabet=st.characters(categories=("Ll", "Lu", "Nd")),  
    min_size=1,  
    max_size=20,  
)  
  
@given(identifiers)  
def test_identifier_round_trip(identifier):  
    assert decode(encode(identifier)) == identifier
```

Écarter un cas invalide

```
from hypothesis import assume  
  
@given(st.integers(), st.integers())  
def test_division_identity(a, b):  
    assume(b != 0)  
    quotient, remainder = divmod(a, b)  
    assert a == quotient * b + remainder
```

`assume` rejette les exemples qui ne respectent pas la précondition. Si les rejets deviennent nombreux, mieux vaut construire directement des valeurs valides.

Exemples et propriétés se complètent

Exemples	Propriétés
communiquent une règle concrète	explorent de nombreuses variations
ciblent une régression connue	découvrent des cas inattendus
donnent un résultat exact	expriment une règle générale
sont choisis par l'humain	sont générés puis réduits

Conserver des exemples lisibles pour les règles importantes, les cas limites et les régressions connues.
Ajouter des propriétés lorsque le problème s'y prête.

Quand chercher une propriété ?

Bon terrain

- encodeur et décodeur
- tri et normalisation
- règles stables sur une structure
- opérations inverses
- calculs sur de grands domaines

Moins naturel

- scénario métier très narratif
- sortie difficile à caractériser
- effets externes dominants
- oracle aussi complexe que le code

La difficulté intéressante est souvent de formuler ce qui doit toujours être vrai.

Pause

On reprend avec le design testable.

DESIGN TESTABLE

Concevoir un code facile à tester

Rendre les dépendances contrôlables et les effets observables.

Ce code fait trop de choses à la fois

```
def send_due_reminders():  
    now = datetime.now()  
    users = requests.get(USERS_URL).json()  
  
    for user in users:  
        if user["deadline"] <= now.isoformat():  
            smtp = smtplib.SMTP("mail.example.org")  
            smtp.sendmail(FROM, user["email"], "Deadline reached")
```

Temps réel

Impossible de choisir facilement l'instant.

Réseau réel

Le test dépend d'un service distant.

Effet réel

Le test risque d'envoyer un courriel.

Les symptômes d'un design difficile à tester

- objets créés profondément dans le code
- état global mutable
- temps, hasard ou environnement lus directement
- calcul et entrées-sorties entremêlés
- méthodes longues avec plusieurs décisions
- résultats observables uniquement par effet externe
- interfaces très larges
- tests exigeant une infrastructure complète

Un test pénible ne signifie pas automatiquement que le design est mauvais. C'est un signal à examiner.

Séparer la sélection de l'effet

Sélectionner

```
def due_recipients(users, now):  
    return [  
        user.email  
        for user in users  
        if user.deadline <= now  
    ]
```

Notifier

```
def notify(recipients, notifier):  
    for recipient in recipients:  
        notifier.send(recipient)
```

La règle de sélection peut être testée directement. L'envoi reste regroupé derrière le notificateur.

Injecter ce qui varie

```
class ReminderService:
    def __init__(self, clock, repository, notifier):
        self.clock = clock
        self.repository = repository
        self.notifier = notifier

    def send_due_reminders(self):
        users = self.repository.list_users()
        recipients = due_recipients(users, self.clock.now())
        notify(recipients, self.notifier)
```

Contrôler

Le test choisit l'heure et les utilisateurs.

Observer

Le notificateur rend l'effet visible.

Remplacer

Les dépendances externes peuvent avoir un double.

Le même constructeur, des objets différents

En production

```
service = ReminderService(  
    clock=SystemClock(),  
    repository=HttpUserRepository(API_URL),  
    notifier=SmtplibNotifier(MAIL_SERVER),  
)
```

Dans un test

```
notifier = RecordingNotifier()  
service = ReminderService(  
    clock=FixedClock(NOW),  
    repository=InMemoryUsers([ADA, LINUS]),  
    notifier=notifier,  
)
```

Le service ne change pas. Le test lui fournit simplement des dépendances contrôlées.

Ce que la nouvelle structure rend explicite

Avant	Après
temps lu directement	horloge fournie au service
réseau mélangé à la logique	repository à la frontière
envoi difficile à observer	notificateur remplaçable
sélection et envoi entremêlés	sélection séparée de l'effet

Le but n'est pas d'ajouter des interfaces partout. Le but est de rendre explicites les variations importantes.

Quelques heuristiques utiles

1

Purifier

Extraire les décisions déterministes quand c'est naturel.

2

Injecter

Rendre contrôlables le temps, le hasard et les services externes.

3

Réduire

Préférer de petites interfaces centrées sur un besoin.

4

Observer

Retourner un résultat ou exposer un effet pertinent.

5

Regrouper

Garder l'infrastructure aux frontières.

6

Résister

Ne pas concevoir uniquement pour satisfaire les tests.

DOUBLES DE TEST

Remplacer une dépendance, avec une raison précise

Stubber un résultat, observer un appel, simuler une panne.

Pourquoi remplacer une dépendance ?

Contrôle

Choisir une réponse rare ou difficile.

Vitesse

Éviter une infrastructure coûteuse.

Sécurité

Ne pas déclencher un effet réel.

Diagnostic

Localiser la responsabilité testée.

On remplace la dépendance située à la frontière, pas la frontière elle-même.

Un vocabulaire pragmatique

Double	Rôle principal
Dummy	remplir un paramètre non utilisé
Stub	fournir une réponse déterminée
Spy	enregistrer ce qui s'est passé
Mock	configurer et vérifier des interactions
Fake	fournir une implémentation simplifiée mais fonctionnelle

Dans les conversations, « mock » est souvent utilisé pour tous ces objets.

Stub : imposer une réponse

```
from unittest.mock import Mock

def test_uses_the_current_time():
    clock = Mock()
    clock.now.return_value = datetime(2026, 10, 12, 9, 0)

    result = greeting(clock)

    assert result == "Bonjour"
```

`return_value` contrôle l'état du monde vu par le code testé.

Mock : vérifier une interaction utile

```
def test_sends_a_notification_to_the_recipient():  
    notifier = Mock()  
  
    notify(["ada@example.org"], notifier)  
  
    notifier.send.assert_called_once_with("ada@example.org")
```

Pertinent

L'appel au notificateur est l'effet promis.

Fragile

Vérifier chaque appel interne imposerait l'algorithme.

Simuler une erreur difficile à provoquer

```
def test_reports_repository_unavailability():  
    repository = Mock()  
    repository.list_users.side_effect = ConnectionError("offline")  
  
    service = ReminderService(CLOCK, repository, NOTIFIER)  
  
    with pytest.raises(ServiceUnavailable):  
        service.send_due_reminders()
```

`side_effect` fait ici lever l'exception choisie lorsque le repository est appelé.

Fake : une implémentation simplifiée

Fake en mémoire

```
class InMemoryUsers:
    def __init__(self, users=()):
        self.users = list(users)

    def list_users(self):
        return list(self.users)
```

Test

```
def test_loads_users_from_repository():
    repository = InMemoryUsers([ADA])

    users = repository.list_users()

    assert users == [ADA]
```

Plus réaliste qu'un stub, mais aussi plus coûteux à maintenir fidèle.

patch vise le nom utilisé par le module

```
# reminders.py
from clock import current_time

def greeting():
    return "Bonjour" if current_time().hour < 18 else "Bonsoir"

# test_reminders.py
from unittest.mock import patch

@patch("reminders.current_time")
def test_evening_greeting(current_time):
    current_time.return_value = datetime(2026, 10, 12, 20, 0)
    assert greeting() == "Bonsoir"
```

reminders.py importe current_time. Le code appelle donc sa référence locale : reminders.current_time.

Trop de mocks fragilisent le test

Signaux

- un mock par objet interne
- une longue séquence d'appels imposée
- un test qui reproduit l'implémentation
- des changements internes qui cassent tout

Réponses possibles

- tester un comportement plus large
- utiliser un fake simple
- déplacer la frontière
- revoir les responsabilités

Mocker les frontières que l'on contrôle mal, pas chaque collaboration que l'on possède.

PERFORMANCE

Mesurer un comportement non fonctionnel

Rapide pour quelle charge, sur quel environnement, avec quel budget ?

Une mesure sans contexte dit peu de choses

Charge

Taille, forme et volume des entrées.

Métrique

Durée, débit, mémoire ou latence.

Environnement

Machine, versions et processus concurrents.

Résultat = scénario + mesure + contexte

D'abord vérifier la correction. Ensuite mesurer la performance du même comportement.

Séparer les tests longs avec un marqueur

Test marqué

```
@pytest.mark.performance
def test_processing_large_dataset():
    data = make_dataset(size=100_000)
    start = perf_counter()
    process(data)
    elapsed = perf_counter() - start
    assert elapsed < BUDGET
```

Exécution

```
$ pytest -m "not performance"
$ pytest -m performance
```

La boucle quotidienne reste rapide. Les mesures coûteuses restent accessibles.

Réduire le bruit de mesure

À faire

- répéter et observer une distribution
- séparer préparation et opération mesurée
- stabiliser les données d'entrée
- comparer sur un environnement similaire

À éviter

- conclure sur une seule exécution
- utiliser un seuil arbitraire très serré
- mélanger réseau et calcul sans le dire
- optimiser avant d'avoir mesuré

Un test de performance instable crée du bruit au lieu de produire une alerte.

COUVERTURE

Qu'avons-nous exécuté ?

Coverage.py mesure le passage des tests dans le code.

La couverture répond à une question limitée

Elle montre

- les lignes exécutées
- les branches parcourues
- les zones jamais atteintes
- l'évolution d'un angle mort

Elle ne montre pas

- la pertinence des assertions
- les exigences oubliées
- la qualité des données
- l'absence de défaut

Un test sans assertion peut augmenter la couverture sans vérifier le moindre résultat.

Produire un rapport avec coverage.py

```
$ coverage run -m pytest  
$ coverage report -m  
$ coverage html
```

Terminal

Vue rapide avec les lignes manquantes.

HTML

Exploration détaillée fichier par fichier.

`coverage run --branch -m pytest` active aussi la mesure des branches.

Ligne couverte, décision incomplète

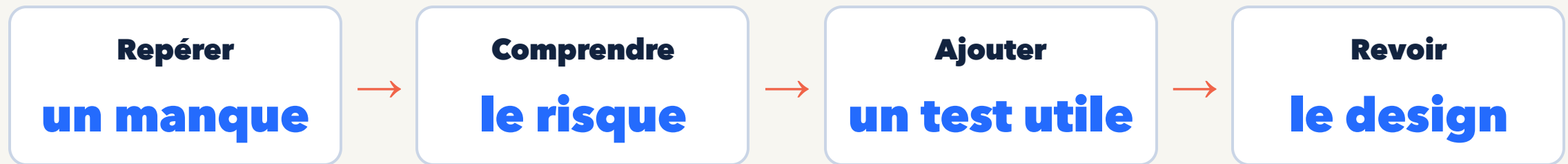
```
def label(age):  
    return "adult" if age >= 18 else "minor"
```

```
def test_adult_label():  
    assert label(20) == "adult"
```

Couverture des lignes : 100 %
Branche `age < 18` : jamais vérifiée

La couverture de branches révèle des décisions non explorées.

Utiliser la couverture comme une carte



Viser une couverture élevée peut être raisonnable. Optimiser uniquement le nombre ne l'est pas.

QUALITÉ DU CODE

Analyse statique et formatage

Automatiser des vérifications sans exécuter le programme.

Le linting analyse le code

```
$ ruff check .  
app.py:1:8: F401 `os` imported but unused  
app.py:5:5: D103 Missing docstring in public function  
Found 2 errors.
```

Erreurs probables

Noms inconnus, imports inutilisés, constructions suspectes.

Cohérence

Imports, conventions et règles partagées.

Feedback rapide

Le diagnostic arrive avant même de lancer les tests.

Dans ce cours, Ruff fournit le linter avec la commande `ruff check`.

Le formatage homogénéise la présentation

Vérifier

```
$ ruff format --check .  
Would reformat: app.py
```

Appliquer

```
$ ruff format .  
1 file reformatted
```

Linting signale des problèmes potentiels. **Formatage** réécrit automatiquement la mise en forme.

Ruff dans le projet pratique actuel

Demandé

`ruff check` doit terminer sans diagnostic.

La cible locale est `make lint`.

Complément utile

`ruff format` peut homogénéiser la mise en forme.

Il ne remplace pas le lint.

Automatiser les conventions libère la revue pour les décisions de conception.

DOCUMENTATION

Documenter et vérifier la documentation

Contrôler les docstrings, puis générer une vue HTML.

Une docstring complète le type et le nom

```
def decode_points(payload: bytes) -> list[tuple[float, float]]:
    """Decode points from their compact binary representation.

    Args:
        payload: Bytes containing the header and point coordinates.

    Returns:
        Decoded two-dimensional points.

    Raises:
        ValueError: If the payload is incomplete or inconsistent.
    """
```

Expliquer les contraintes et les erreurs utiles, pas traduire chaque ligne de code.

Vérifier puis générer la documentation

Vérifier

```
$ ruff check --select D .  
All checks passed!
```

Automatique

Présence, conventions, imports et génération.

Générer

```
$ pdoc --html my_package \  
--output-dir docs
```

Revue humaine

Exactitude, clarté et utilité du contenu.

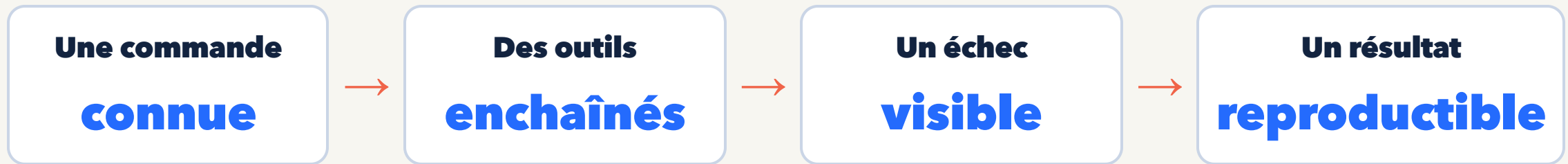
Le package s'appelle `pdoc3`. La commande installée s'appelle `pdoc`.

BOUCLE LOCALE

Automatiser la vérification locale

Une commande reproductible doit donner le même résultat à toute l'équipe.

Ce que l'on veut obtenir



La vérification locale doit être simple à lancer et difficile à oublier.

Make organise les commandes du projet

- `make unit_test` pour les tests rapides
- `make perf_test` pour les mesures longues
- `make test` pour l'ensemble des tests
- `make coverage` pour le rapport de couverture
- `make lint` pour l'analyse statique
- `make doc` pour la documentation HTML

Make ne réalise pas lui-même ces vérifications. Il leur donne des noms communs et les organise.

Quand exécuter quoi ?

Moment	Vérification utile
Après une petite modification	le test concerné, par exemple <code>pytest tests/test_calendar.py::test_name</code>
Régulièrement	les tests unitaires et le lint
Avant de partager	la suite complète et la couverture
Avant un jalon	performance et documentation

Un serveur de CI pourrait plus tard exécuter les mêmes commandes après chaque partage.

TECHNIQUES DE TESTS

Questions ?

Points à clarifier, cas particuliers, retours d'expérience.